

Academic Year: 2026-27

Group No.

69

PUNE INSTITUTE OF COMPUTER TECHNOLOGY

Dhankawadi, Pune -43.



DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION ENGINEERING (E&TCE)

A

SYNOPSIS

ON

**“Implementation and Performance Analysis of
RAG vs. Fine-Tuned LLMs for Cyber Threat
Mapping”**

Project by

Project Group Members:

Roll No	Name	Preferred Guides
42373	Soham Kharat	Dr. A. V. Rajput
42353	Aryan Mashalkar	Mr. N. G. Nirmal
42430	Anand Pandit	Ms. S. M. Hosamani

Academic Year: 2026-27

- ❖ **AIM OF PROJECT:** - To perform a comparative analysis of accuracy and latency trade-offs between Retrieval-Augmented Generation (RAG) pipelines and Parameter-Efficient Fine-Tuning (PEFT) approaches for mapping unstructured cyber threat intelligence to the MITRE ATT&CK framework

❖ **INTRODUCTION :-**

Manual threat attribution is highly complex and time-consuming for Security Operations Center (SOC) analysts dealing with dynamic threat environments, but Large Language Models (LLMs) offer significant potential for automation by processing Tactics, Techniques, and Procedures (TTP) descriptions. Recent post-2023 literature on LLM-assisted attack attribution centers on comparative performance, operational limits around latency, and remaining research gaps for SOC deployment. Within this context, cybersecurity research suggests that Retrieval-Augmented Generation (RAG) often outperforms fine-tuning alone when tasks require current, external, or highly structured security knowledge, though this advantage relies heavily on retrieval quality since missing or noisy context can lower precision even if recall remains strong. Conversely, parameter-efficient fine-tuning remains highly competitive for constrained environments; for example, OpenSOC-AI demonstrated that LoRA tuning of a 1.1B model on 450 SOC examples completed in under five minutes on a single T4 GPU. Building on these findings, this project hypothesizes that RAG and fine-tuning offer distinctly different trade-offs in accuracy and computational overhead, and will systematically map these performance metrics to determine optimal SOC deployment strategies.

HARDWARE & SOFTWARE REQUIREMENTS:-

➤ **HARDWARE REQUIRED:-**

Part/Environment	Technical Specification	Purpose
Cloud Compute (Training)	NVIDIA A100 (80GB VRAM) / T4 (16GB)	Used for parallel execution of

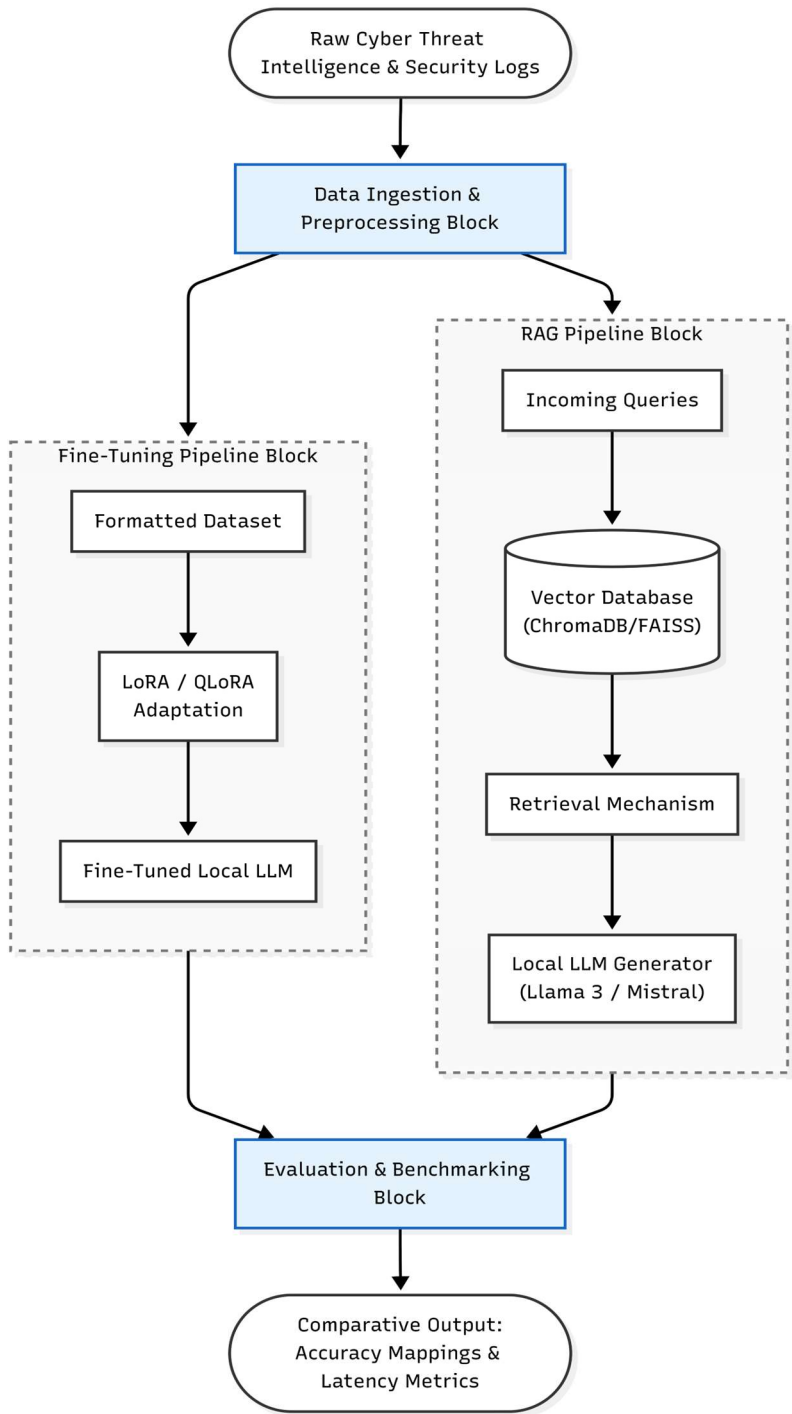
Academic Year: 2026-27

	VRAM) via Cloud resources	LoRA/QLoRA parameter-efficient
Local Compute (Inference & RAG)	Dedicated GPU (NVIDIA RTX 3050, 6GB GDDR6 VRAM)	Used to host quantized local models (e.g., Llama 3 8B 4-bit GGUF) for running RAG pipelines and testing inference latency.
System Memory	32 GB DDR4/DDR5 RAM	Manages vector database chunking, document parsing, and CPU offloading workflows.

➤ SOFTWARE REQUIRED:-

1. **Local LLMs:** Meta Llama 3 (8B / 3B) & Mistral (7B) Instruct Models (Quantized GGUF formats for local inference).
2. **Programming Language:** Python 3.10+ with Conda Environment management.
3. **Deep Learning Framework:** PyTorch 2.x (compiled with CUDA 12.x backend support).
4. **Fine-Tuning Frameworks:** PEFT, BitsAndBytes (for 4-bit/8-bit double quantization configurations), and Hugging Face Transformers.
5. **RAG Orchestration:** LangChain / LlamaIndex framework architectures.
6. **Vector Store:** ChromaDB / FAISS for semantic indexing and hierarchical context retrieval.

❖ **BLOCK DIAGRAM OF PROJECT: -**



Academic Year: 2026-27

❖ DESCRIPTION OF BLOCK DIAGRAM :-

- Raw CTI & Security Logs: Ingests raw threat reports and machine-generated security log data for downstream attribution.
- Data Ingestion & Preprocessing: Cleans, tokenizes, and chunks raw text data into standardized formats for the downstream pipelines.
- RAG Pipeline Block: Fetches contextual MITRE ATT&CK reference data from a vector database to guide local LLM threat mapping.
- Fine-Tuning Pipeline Block: Modifies foundational LLM weights directly using parameter-efficient LoRA/QLoRA training on domain-specific threat datasets.
- Evaluation & Benchmarking Block: Compares the parallel architectures head-to-head on classification accuracy, memory usage, and execution latency.
- Comparative Output: Outputs quantitative trade-off reports to identify the optimal configuration for real-time SOC deployment.

❖ STEP- WISE EXECUTION:-

STEP 1 (July)	▪ Finalization of project domain and project Topic. ▪ Design of block diagram and plan for project Execution. ▪ Preparation of synopsis
STEP 2 (August)	▪ Dataset curation of unstructured CTI and structured security logs for MITRE ATT&CK mapping
STEP 3 (September)	▪ Setup of local computational environment with NVIDIA GPU support. ▪ Development of the RAG-based pipeline using vector databases and LangChain..
STEP 4 (October)	▪ Implementation of the LoRA/QLoRA fine-tuning approach on baseline Llama 3/Mistral models using PyTorch.
STEP 5 (December)	▪ Testing and integration of both pipelines into a unified evaluation framework. ▪ First Assessment submission.
STEP 6 (Jan-Feb)	▪ Benchmarking system performance, specifically focusing on the latency, recall, and precision trade-offs between the two architectures.
STEP 7	▪ Submission of second Assessment.

Academic Year: 2026-27

(Feb-March)	▪ Optimization of pipelines to reduce computational overhead and improve inference speeds.
STEP 8 (April)	▪ Preparation and writing of final project report. ▪ Checking and Printing of project report.
STEP 9 (May)	▪ Submission of final project report.

REFERENCES:-

- ❖ Fayyazi, R., Taghdimi, R., & Yang, S. (2023). Advancing TTP Analysis: Harnessing the Power of Large Language Models with Retrieval Augmented Generation. *2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)*, 255-261.
 - <https://doi.org/10.1109/acsacw65225.2024.00036>
- ❖ Krishna, V. (2024). AttackQA: Development and Adoption of a Dataset for Assisting Cybersecurity Operations using Fine-tuned and Open-Source LLMs.
 - *ArXiv, abs/2411.01073*. <https://doi.org/10.48550/arxiv.2411.01073>
- ❖ Lakatos, R., Pollner, P., Hajdu, A., & Joó, T. (2024). Investigating the performance of Retrieval-Augmented Generation and fine-tuning for the development of AI-driven knowledge-based systems.
 - *ArXiv, abs/2403.09727*. <https://doi.org/10.3390/make7010015>